Project Development Phase
Performance Test

Date	01.04.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. Enforcing Mandatory Fields with UI Policies

UI Policies are client-side logic used to set fields as **Mandatory**, **Read-only**, or **Visible** based on specific conditions without requiring scripting.

Component	Description
Conditions	The "When" (e.g., `State is Resolved` or `Priority is 1`).
UI Policy Actions	The "What" (e.g., Set `Close Notes` to Mandatory = True).
On Load	Determines if the policy runs when the form first opens.
Reverse if False	If the conditions are no longer met, the action is undone automatically.

Best Practices:

- **Order Matters:** Use the **Order** field to manage conflicting policies (lower numbers run first).
- **Keep it Simple:** Avoid complex scripts in the "Execute if true" tab if a simple UI Policy Action can achieve the goal.

2. Managing Changes with Update Sets

Update Sets capture configuration changes (metadata) so they can be moved from a Development instance to Test or Production.

The Workflow Cycle

- **Create/Select:** Navigate to **Local Update Sets** and create a new set. Set it to "Current."
- **Configure:** Create your UI Policies and Actions. ServiceNow automatically records these entries.
- **Validate:** Review the **Customer Updates** related list to ensure your UI Policy and UI Policy Actions are captured.
- **Complete:** Change the State to **Complete**.
- **Export/Retrieve:** On the target instance (e.g., Test), "Retrieve" the update set, "Preview" it to check for conflicts, and finally "Commit."

3. Comparison: UI Policies vs. Data Policies

While UI Policies handle the form experience, they do not enforce rules during data imports or Web Service calls.

Feature	UI Policy	Data Policy
Execution	Client-side (Browser)	Server-side (Database)
Mandatory Check	Only on the Form	Form, Import Sets, & APIs
Scripting	Supports UI Scripts	No Scripting
User Experience	Instant visual feedback	Error message upon save

4. Migration Troubleshooting (The "Preview" Phase)

Before committing an Update Set, ServiceNow runs a **Preview** to identify collisions.

- **Collisions:** Occur if someone else modified the same UI Policy in the target instance.
- **Missing Dependencies:** Occur if your UI Policy references a field that doesn't exist in the target instance yet.
- **Resolution:** You must choose to "Accept Remote Update" (overwrite the target) or "Skip Remote Update" (keep the target as is).

Note: Always ensure your **Update Set** is marked as "Current" before you start building your UI Policy, or your changes will be recorded in the "Default" set, making migration difficult.